

Accélération d'une FFT sur processeur RISC-V CVA6 : Optimisations Matérielles et Micro-Architecturales

Vitória Júlia AGUIAR LIBERATO
CentraleSupélec
Gif-Sur-Yvette, France

Fares BALY
CentraleSupélec
Gif-Sur-Yvette, France

Hector BRIENT
CentraleSupélec
Gif-Sur-Yvette, France

Benoit MASSENET
CentraleSupélec
Gif-Sur-Yvette, France

Résumé—Cet article présente l'accélération d'une FFT de 512 points complexes 16 bits sur un cœur RISC-V CV32A6 implémenté sur FPGA Zybo Z7-20. Notre solution combine un coprocesseur connecté à l'interface CV-X-IF, dédié aux opérations radix-2 et radix-4, et un allègement du cœur exploitant le profil restreint de l'application. Le coprocesseur enchaîne les radix-4 successifs en quatre instructions par itération, valeur minimale imposée par la contrainte d'une seule sortie par instruction de l'ISA RISC-V. Un format de données complexes empaqueté sur 32 bits divise par deux le nombre de transferts entre le cœur et le coprocesseur. Combinées au retrait des blocs inutilisés (MMU, PMP, extension atomique) et à plusieurs ajustements micro-architecturaux, ces modifications réduisent le calcul de la FFT de 153 490 à 42 729 cycles en simulation, soit une accélération d'un facteur 3,59, au prix d'une baisse de 19 % de la fréquence maximale, dans la limite des 20 % autorisés par le règlement.

I. INTRODUCTION

Ce projet s'inscrit dans le cadre du 6ème concours national étudiant RISC-V [1]. L'objectif principal de cette édition était d'accélérer le calcul d'une transformée de Fourier (FFT) sur un cœur de processeur RISC-V CV32A6 [2], déployé sur le FPGA d'une carte Zybo Z7-20.

La FFT est effectuée par un programme en langage C traitant un tableau de 512 valeurs complexes, sous forme d'entiers de 16 bits. Les critères d'évaluation imposaient un fonctionnement validé sur carte, la non-régression de l'architecture (validation avec CoreMark et MNIST), et l'obtention de résultats exacts au bit près. Notre méthodologie a reposé sur un co-design matériel/logiciel rigoureux, respectant l'interdiction de concevoir un accélérateur autonome, ce qui a conduit au développement d'un coprocesseur dédié aux radix-2 et radix-4, intégré au pipeline du processeur central via l'interface CV-X-IF [3].

Le tableau I récapitule les caractéristiques du projet de référence fourni par les organisateurs, qui constitue le point de comparaison des résultats présentés dans cet article.

II. CONCEPTION DU COPROCESSEUR MATÉRIEL VIA CV-X-IF

Le cœur CV32A6 inclut l'interface CV-X-IF, permettant l'ajout de coprocesseurs capables d'exécuter des instructions personnalisées. Nous l'avons mise à profit pour paralléliser les calculs des opérations de radix-2 et radix-4 au sein du coprocesseur. Cette approche s'intègre naturellement à l'algorithme

TABLE I
MÉTRIQUES DE LA CONFIGURATION DE RÉFÉRENCE

Métrique	Valeur
Cycles FFT-512 (simulation)	153 490
Instructions FFT-512 (simulation)	124 142
Fréquence maximale post P&R	40.15 MHz
LUTs (top-level)	19 810
Flip-Flops (top-level)	14 624
RAMB36	60
DSP Blocks	4

de la bibliothèque kiss_fft [4] fournie par les organisateurs, qui décompose le calcul en une succession d'étages de radix-2 et radix-4.

La figure 1 donne une vue d'ensemble de l'architecture du coprocesseur et de ses connexions au cœur.

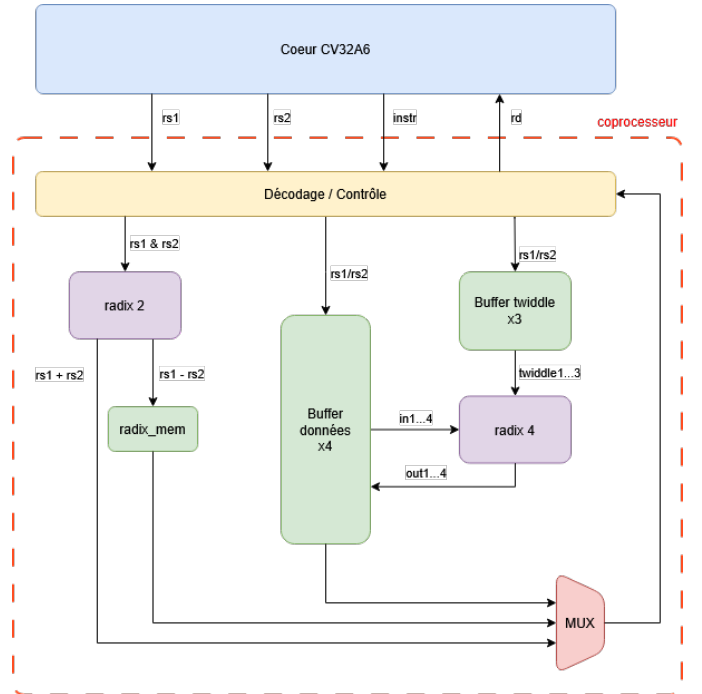


FIGURE 1. Schéma de l'architecture globale du coprocesseur

A. Architecture du coprocesseur et jeu d'instructions

Le coprocesseur repose sur deux composants internes principaux : un buffer de données et un buffer de coefficients. Ces éléments permettent d'accumuler les opérandes sur plusieurs instructions successives avant de déclencher un calcul. Ce mécanisme pallie les limitations de l'ISA qui restreint chaque instruction à deux registres d'entrée et un de sortie, rendant impossible la transmission simultanée des multiples opérandes requis par les opérations complexes.

Le tableau II résume le jeu d'instructions personnalisées défini pour le coprocesseur afin d'exploiter cette architecture.

TABLE II
JEU D'INSTRUCTIONS PERSONNALISÉES DU COPROCESSEUR CV-X-IF

Instruction	rs1	rs2	rd
<i>Radix-2</i>			
<code>coproc_radix_c</code>	<code>in_a</code>	<code>in_b</code>	$a + b$
<code>coproc_radix_r</code>	—	—	$a - b$
<code>coproc_r4_push_2in</code>	<code>in1</code>	<code>in2</code>	—
<code>coproc_r4_push_mult</code>	<code>in3</code>	<code>in4</code>	— (lance le calcul)
<code>coproc_r4_push_read_2</code>	<code>in2_{k+1}</code>	<code>coeff2_{k+1}</code>	<code>out2_k</code>
<code>coproc_r4_push_read_3</code>	<code>in3_{k+1}</code>	<code>coeff3_{k+1}</code>	<code>out3_k</code>
<code>coproc_r4_push_read_4</code>	<code>in4_{k+1}</code>	<code>coeff4_{k+1}</code>	<code>out4_k</code>
<code>coproc_r4_push_read_1</code>	<code>in1_{k+1}</code>	—	<code>out1_k</code> (lance le calcul)

Pour le radix-2, la contrainte d'entrée/sortie est modérée. Une première instruction `coproc_radix_c(a,b)` envoie les deux entrées, déclenche le calcul, et retourne $a + b$ directement dans `rd`. La valeur $a - b$ est simultanément stockée dans un registre interne `radix_mem` du coprocesseur. Une seconde instruction `coproc_radix_r()` la lit sans arguments supplémentaires. La figure 2 illustre ce flot.

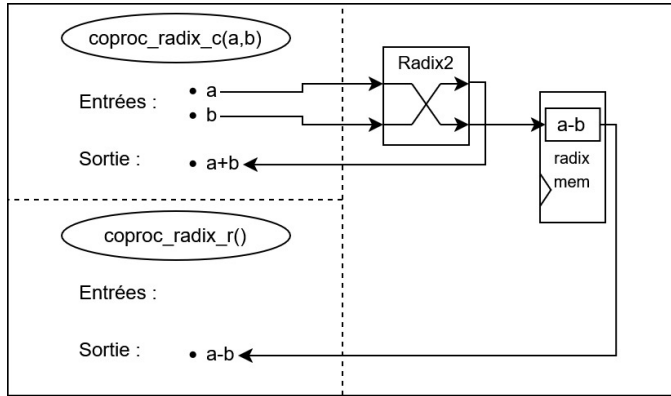


FIGURE 2. Architecture de l'opération radix-2

B. Pipelining des opérations Radix-4

L'opération radix-4 nécessite sept entrées (en incluant les "twiddle factors") et génère quatre sorties. Étant donné la contrainte d'un registre de sortie par instruction, le nombre minimum de cycle requis pour l'exécution d'un radix-4 est de quatre. Pour se rapprocher au maximum de cette limite théorique, et puisque selon l'algorithme de kissFFT les radix-4 sont appelés en série, une version pipelinée du radix-4 a été implémentée.

Ainsi, on procède en simultané à la récupération des données de sorties du radix-4 n et au chargement des opérandes du radix-4 $n + 1$. En régime stationnaire, c'est à dire après que le premier radix-4 de la série ait été calculé, une boucle de quatre instructions est répétée, qui récupère successivement les quatre sorties d'un radix-4, tout en transférant les entrées et coefficients du radix-4 suivant. Cette approche permet, à une initialisation près, d'exécuter un radix-4 en quatre instructions, et donc d'atteindre la limite. Nous détaillons dans la suite de cette sous-partie les différentes instructions mettant en place ce pipelining.

La figure 3 représente l'initialisation de la chaîne de radix-4. Une particularité de l'algorithme utilisé par kissFFT est que le premier radix-4 de chaque série utilise uniquement des twiddle factors égaux à un. Cela signifie qu'il n'est pas nécessaire de les charger dans le coprocesseur, et que l'initialisation peut être réalisée en seulement deux instructions. Ces dernières permettent chacune de charger deux des quatre opérandes du radix-4 dans le buffer. La deuxième a en plus pour effet de déclencher le calcul du radix-4, dont les résultats sont immédiatement stockés dans le buffer à la place des opérandes.

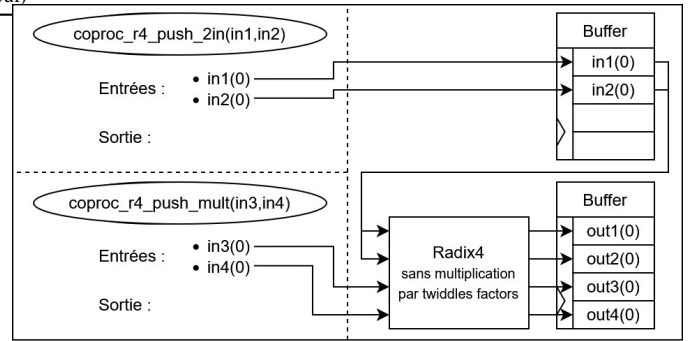


FIGURE 3. Initialisation d'une série de radix-4

Une fois la série initialisée, les mêmes étapes se répètent jusqu'à sa fin. Chaque instruction récupère une des sorties du radix-4 précédent, actuellement stockée dans le buffer, et écrit à sa place une des valeurs d'entrée du prochain radix-4. Elle charge aussi le twiddle factor associé dans un buffer séparé. La quatrième et dernière instruction est une exception : elle charge le premier opérande du radix-4, dont le twiddle factor associé est toujours de un et n'a donc pas besoin d'être chargé. Cette instruction est aussi responsable du déclenchement du calcul du radix-4. Les figures 4, 5, 6 et 7, présentent cette boucle de manière schématique.

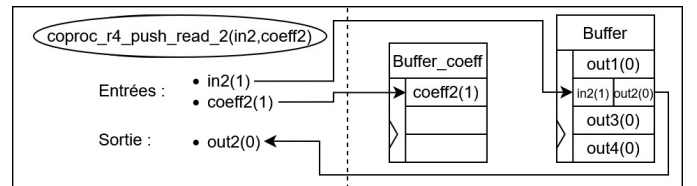


FIGURE 4. Deuxième instruction de la boucle pipelinée pour le radix-4

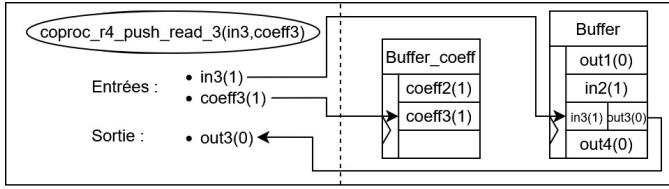


FIGURE 5. Troisième instruction de la boucle pipelinée pour le radix-4

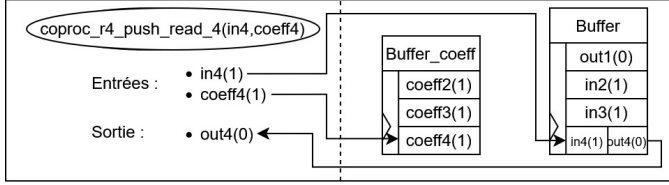


FIGURE 6. Quatrième instruction de la boucle pipelinée pour le radix-4

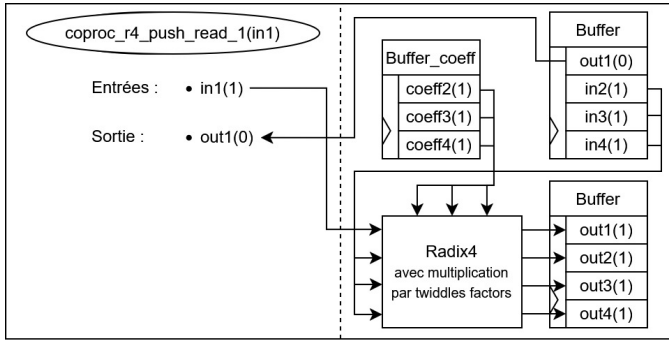


FIGURE 7. Dernière instruction de la boucle pipelinée pour le radix-4, déclenche le calcul

La figure 8 illustre cet ordonnancement ininterrompu sur 9 instructions consécutives couvrant deux radix-4 successifs. Les cases colorées correspondent au moment où chaque donnée est transmise ou calculée. Dès l'instruction 5, le cœur commence à envoyer les entrées du second radix pendant que les 4 sorties du premier radix sont rapatriées (instructions 5 à 9). Le recouvrement est ainsi total en régime stationnaire : aucun cycle n'est perdu entre deux radix consécutifs.

	Instruction 1	Instruction 2	Instruction 3	Instruction 4	Instruction 5	Instruction 6	Instruction 7	Instruction 8	Instruction 9
Entrée 1 et 2	0	0	0	0	1	1	1	1	2
Entrée 3 et 4	X	0	0	0	0	1	1	1	1
Entrée 5 et 6	X	X	0	0	0	0	1	1	1
Entrée 7	X	X	X	0	0	0	0	1	1
Résultat radix	X	X	X	0	0	0	0	1	1
Sortie 1	X	X	X	X	0	0	0	0	1
Sortie 2	X	X	X	X	X	0	0	0	0
Sortie 3	X	X	X	X	X	X	0	0	0
Sortie 4	X	X	X	X	X	X	X	0	0

FIGURE 8. Ordonnement pipeliné des instructions pour le calcul ininterrompu des radix-4

C. Optimisation du format des données

Dans la bibliothèque kiss_fft [4], les parties réelle et imaginaire d'une donnée complexe, chacune codée sur 16 bits, sont stockées séparément dans deux mots de 32 bits. Ce format est incompatible avec l'architecture de notre coprocesseur, qui nécessite que les deux parties soient transmises simultanément. Pour lever cette incompatibilité, nous avons modifié le type des données complexes dans le code C, en empaquetant les parties réelle et imaginaire dans un unique mot de 32 bits (voir figure 9). Cette modification a également pour effet de diviser par deux le nombre d'accès mémoire (lectures et écritures), la moitié des bits précédemment échangés entre la mémoire et le cœur ne portant aucune information utile.

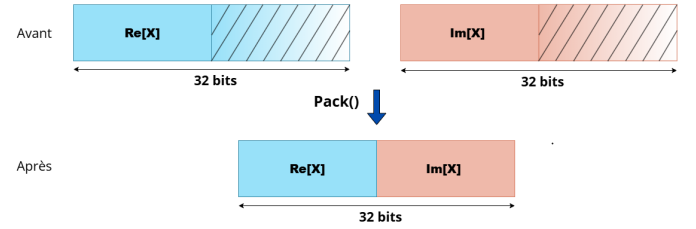


FIGURE 9. Empaquetage des données complexes 16 bits dans un unique mot de 32 bits

III. MODIFICATIONS MICRO-ARCHITECTURALES DU CVA6

L'ajout du coprocesseur a été accompagné d'un nettoyage de l'architecture standard du processeur afin d'éliminer la logique superflue et d'optimiser les chemins d'exécution.

A. Suppression des blocs inutilisés

L'application cible ne nécessitant pas de système d'exploitation avancé, de nombreux mécanismes matériels ont été désactivés :

- **Gestion mémoire et MMU/PMP** : La logique matérielle associée à la MMU et au mécanisme PMP (Physical Memory Protection) a été enlevée, libérant le chemin critique dans l'unité Load/Store (LSU).
- **Atomics** : L'extension d'instructions atomiques (inutile dans un contexte purement séquentiel) a été intégralement retirée, économisant près de 1295 LUTs.
- **CSR Regfile** : L'essentiel des compteurs de performance matériels étendus et registres d'états inexploités ont été supprimés de la synthèse.

B. Réglages architecturaux et compromis

a) **Scoreboard**: La capacité du scoreboard a été doublée, passant de 4 à 8 entrées. Bien que cette modification augmente la logique du module `issue_stage`, qui passe de 2272 LUTs et 798 registres à respectivement 4396 et 1432, elle offre la flexibilité requise au cœur pour absorber les latences combinatoires et conserver davantage d'instructions en vol.

b) Pipeline de Chargement (LoadPipeRegs):

Le retrait de l'étage de pipeline situé sur le chemin de retour des opérations de lecture mémoire (CVA6ConfigNrLoadPipeRegs = 0) a drastiquement réduit la latence cyclique des chargements, environ 2000 cycles. Ce choix ajoute une pression sur le timing, mais le rapport de post-placement-routage confirme un "slack" maîtrisé de +0.480 ns à la fréquence cible requise, permettant ainsi la réduction des cycles d'exécution en garantissant une bonne fiabilité.

Ces modifications n'ont pas seulement simplifié certains blocs du cœur, elles ont aussi déplacé le compromis global entre surface, fréquence et nombre de cycles. Les résultats post-implémentation permettent précisément d'en mesurer l'effet.

IV. RÉSULTATS MATÉRIELS FINAUX

Les modifications décrites dans la section précédente ont été comparées avec la référence après synthèse et place & route. Le tableau III en présente les résultats. Le nombre de cycle de la FFT passe de 153 490 à 42 729, soit une réduction de 72.16 %, accompagné d'une diminution équivalente du nombre d'instructions de 124 142 à 33 661. La fréquence maximale baisse de 19 %, en deçà de la limite imposée de 20 %. L'impacte sur les ressources reste raisonnable : hausse modérée des LUTs et des FFs, plus marquée pour les DSP en raison des multiplieurs de twiddles intégrés au coprocesseur.

TABLE III

COMPARAISON DES RÉSULTATS DE SYNTHÈSE ET DE SIMULATION ENTRE LA VERSION DE RÉFÉRENCE ET LE DESIGN FINAL

Métrique	Référence	Design final	Δ
Cycles FFT-512	153 490	42 729	-72.16 %
Instructions FFT-512	124 142	33 661	-72.89 %
Fréquence maximale (MHz)	40,15	32,50	-19.05 %
Total LUTs	19 810	21 031	+6.16 %
Logic LUTs	19 570	20 789	+6.23 %
LUTRAMs	200	202	+1.00 %
SRLs	40	40	0.00 %
FFs	14 624	15 013	+2.66 %
RAMB36	60	60	0.00 %
RAMB18	5	5	0.00 %
DSP Blocks	4	19	+375.00 %

Le tableau IV confirme que la fréquence reste dans la limite autorisée. Il s'agit bien d'un compromis souhaitable, puisque la baisse de la fréquence nous offre le slack nécessaire à l'exécution en un cycle d'un radix-4 complet dans notre coprocesseur.

TABLE IV

ANALYSE DE TIMING POST-IMPLÉMENTATION (VIVADO, CORNER SLOW)

Paramètre	Référence	Design final
Période cible (ns)	25.000	31.250
Slack chemin critique (ns)	+0.094	+0.480
Fréquence maximale (MHz)	40.15	32.50
Variation vs. référence	—	-19.0 %

Afin de distinguer l'effet propre du coprocesseur de celui des optimisations architecturales du cœur, nous considérons trois versions du design : la version *initiale*, la version *Copro*, correspondant à l'intégration du coprocesseur sans optimisation supplémentaire du cœur, et la version *Copro optimisé*, qui ajoute à cette intégration les simplifications et réglages architecturaux décrits précédemment. Le tableau V montre ainsi que les optimisations architecturales ne suppriment pas le coût d'intégration, mais qu'elles le contiennent, ce qui est particulièrement pertinent dès lors que les ressources top-level peuvent servir de critère de départage.

TABLE V

COMPARAISON DES RESSOURCES DU DESIGN CVA6_ZYBO_Z7_20

Ressource	Initiale	Copro	Copro optimisé
Total LUTs	19810	21544	21031
Logic LUTs	19570	21304	20789
LUTRAMs	200	200	202
SRLs	40	40	40
FFs	14624	14956	15013
RAMB36	60	60	60
RAMB18	5	5	5
DSP Blocks	4	19	19

À l'échelle du design complet, l'intégration du coprocesseur augmente naturellement l'occupation logique, en particulier sur les LUTs, les Logic LUTs et les DSP. Les optimisations du cœur permettent ensuite de réduire une partie de ce surcoût entre les versions *Copro* et *Copro optimisé*, surtout du côté des LUTs, tandis que les ressources mémoire dédiées restent globalement stables (tableau V). Autrement dit, ces optimisations ne suppriment pas le coût d'intégration, mais en limitent l'impact, ce qui reste essentiel lorsqu'on cherche à conserver la fréquence la plus élevée possible.

Le tableau VI précise d'où proviennent ces évolutions. Le coût nouveau du bloc `cvxif_example_coprocessor` traduit directement l'accélération matérielle apportée à la FFT, tandis que la disparition de `atomics` et l'allègement du `csr_regfile`, de l'`ex_stage` et de la `lsu` montrent que les optimisations du cœur ont bien servi à compenser une partie du surcoût lié au coprocesseur. À l'inverse, l'augmentation de l'`issue_stage` reflète un choix volontaire d'amélioration du débit, via l'élargissement du scoreboard. Autrement dit, la logique supplémentaire n'a pas été ajoutée de manière uniforme, mais concentrée sur les éléments contribuant le plus directement à la réduction du nombre de cycles.

TABLE VI

ÉLÉMENTS LES PLUS IMPACTÉS PAR L'INTÉGRATION DU COPROCESSEUR ET LES OPTIMISATIONS ARCHITECTURALES

Élément	LUTs			FFs		
	Initiale	Copro	Copro optimisé	Initiale	Copro	Copro optimisé
<code>atomics</code>	965	957	0	463	463	0
<code>cvxif_ex_copro</code>	0	1353	1353	0	298	298
<code>csr_regfile</code>	974	960	103	729	729	487
<code>ex_stage</code>	2367	2440	1226	1116	1120	1067
<code>lsu</code>	1132	1189	531	946	949	896
<code>issue_stage</code>	2272	2463	4396	167	807	1432

D'autres blocs évoluent plus modérément. Le `cache` suit la tendance générale de simplification des chemins mémoire, avec une légère baisse de logique, tandis que le `id_stage` reste globalement stable, ce qui est cohérent avec son rôle central dans le décodage et le contrôle de base. Le `frontend`, en revanche, n'a pas été significativement réduit dans la version finale, car la modification étudiée augmentait trop fortement le nombre de cycles. Cette observation confirme qu'une simplification locale n'est pas nécessairement bénéfique à l'échelle du cœur complet (tableaux VI et III).

Au total, les résultats précédents montrent que l'intégration du coprocesseur, combinée à des optimisations ciblées du cœur, permet d'obtenir un gain important sur la FFT-512 tout en maintenant un coût matériel et temporel compatible avec les contraintes du projet (tableaux IV et III).

V. PISTES D'AMÉLIORATIONS

Les optimisations présentées dans ce papier sont exclusivement matérielles, conformément au règlement du concours. Des optimisations logicielles ont néanmoins été explorées à titre expérimental, et montrent une réduction supplémentaire significative.

La principale piste concerne la réduction de la pression mémoire dans l'algorithme `kiss_fft` [4]. Dans sa version originale, la structure `kiss_fft_state` (contenant les tableaux de facteurs et de twiddle factors) est passée en paramètre à chacun des 512 appels récursifs, générant autant de copies dans la stack et des défauts de cache répétés. Sa globalisation, combinée à la suppression de la variable `in_stride` (toujours égale à 1 mais multipliée à chaque niveau de récursion), réduit les accès au D-cache de plus de 57 % et fait chuter le compteur *Scoreboard Full* de 500 à 55 événements bloquants. Une distinction explicite des deux cas de la récursion ($m = 1$ pour les radix-2 de fond de pile, $m \neq 1$ pour les radix-4) permet en outre d'éliminer des branchements et initialisations de variables inutiles.

L'ensemble de ces modifications logicielles, bien qu'exclues de la soumission officielle, a permis d'atteindre **29 389 cycles** et **22 517 instructions** lors de nos mesures internes.

VI. CONCLUSION

Ce travail montre l'efficacité d'une approche de co-conception matérielle/logicielle directement intégrée dans le pipeline d'un cœur CV32A6. En implémentant un flow pipeline dédié au radix-4 et en remaniant les ressources internes du processeur (via la suppression des modules inutilisés et le redimensionnement du scoreboard), le compromis entre la performance et le respect du timing a été optimisé, nous permettant d'obtenir un résultat final de **42 729 cycles**, soit une diminution de 72.16 % par rapport à la version de référence. Ces changements ne créent pas de régression des capacités du cœur, comme le confirme le bon fonctionnement de CoreMark et MNIST.

REMERCIEMENTS

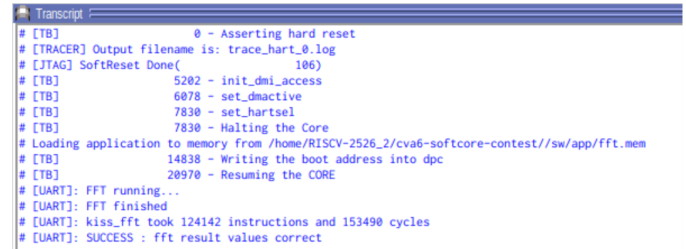
Les auteurs remercient les organisateurs du concours — Thales Research and Technology, le GDR SOC² et le CNFM — pour la mise à disposition du kit, le soutien technique et les nombreuses réponses apportées à nos questions tout au long du projet. Nous remercions également nos encadrants à CentraleSupélec pour leur accompagnement.

RÉFÉRENCES

- [1] Thales Research and Technology, GDR SOC², and CNFM, "6^e concours national étudiant RISC-V 2025–2026," https://gdrsoc2.unistra.fr/media/eventdescription/Annonce_RISC-V_contest_2025-2026_v1.2_IW69SLb.pdf
- [2] OpenHW Group, "CVA6 User Manual — CV32A6 / CV64A6 RISC-V CPU," <https://docs.openhwgroup.org/projects/cva6-user-manual/>
- [3] OpenHW Group, "CV-X-IF Interface and Coprocessor," https://docs.openhwgroup.org/projects/cva6-user-manual/01_cva6_user/CVX_Interface_Coprocessor.html
- [4] M. Borgerding, "Kiss FFT — A mixed-radix Fast Fourier Transform based upon the principle "Keep It Simple, Stupid"," <https://github.com/mborgerding/kissfft>

ANNEXE

Les figures 10, 11 et 12 présentent les sorties de simulation Questa pour la version de référence et les deux versions de nos modifications.

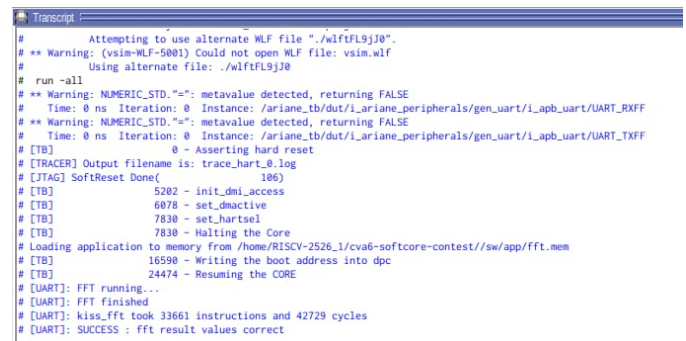


```

Transcript
# [TB] 0 - Asserting hard reset
# [TRACER] Output filename is: trace_hart_0.log
# [JTAG] SoftReset Done( 106)
# [TB] 5202 - init_dmi_access
# [TB] 6078 - set_dmacactive
# [TB] 7830 - set_hartsel
# [TB] 7830 - Halting the Core
# Loading application to memory from /home/RISCV-2526_2/cva6-softcore-contest//sw/app/fft.mem
# [TB] 14838 - Writing the boot address into dpc
# [TB] 20970 - Resuming the CORE
# [UART]: FFT running...
# [UART]: FFT finished
# [UART]: kiss_fft took 124142 instructions and 153490 cycles
# [UART]: SUCCESS : fft result values correct

```

FIGURE 10. Log de simulation Questa - version de référence : 124 142 instructions, **153 490 cycles**



```

Transcript
# Attempting to use alternate WLF file "/wlf1f19j30".
# ** Warning: (vsim-WLF-9001) Could not open WLF file: vsim.wlf
# Using alternate file: ./wlf1f19j30
# run -all
# ** Warning: NUMERIC_STD.": metavalue detected, returning FALSE
# Time: 0 ns Iteration: 0 Instance: /ariane_tb/dut/i_ariane_peripherals/gen_uart/i_apb_uart/UART_RXFF
# ** Warning: NUMERIC_STD.": metavalue detected, returning FALSE
# Time: 0 ns Iteration: 0 Instance: /ariane_tb/dut/i_ariane_peripherals/gen_uart/i_apb_uart/UART_TXFF
# [TB] 0 - Asserting hard reset
# [TRACER] Output filename is: trace_hart_0.log
# [JTAG] SoftReset Done( 106)
# [TB] 5202 - init_dmi_access
# [TB] 6078 - set_dmacactive
# [TB] 7830 - set_hartsel
# [TB] 7830 - Halting the Core
# Loading application to memory from /home/RISCV-2526_1/cva6-softcore-contest//sw/app/fft.mem
# [TB] 16590 - Writing the boot address into dpc
# [TB] 24474 - Resuming the CORE
# [UART]: FFT running...
# [UART]: FFT finished
# [UART]: kiss_fft took 33661 instructions and 42729 cycles
# [UART]: SUCCESS : fft result values correct

```

FIGURE 11. Log de simulation Questa — version finale soumise au concours : 33 661 instructions, **42 729 cycles**

La figure 13 présente le résultat du test sur carte de la version finale de notre architecture.

```

Transcript:
# run -all
# ** Warning: NUMERIC_STD."=": metavalue detected, returning FALSE
# Time: 0 ns Iteration: 0 Instance: /ariane_tb/dut/i_ariane_peripherals/gen_uart/i_apb_uart/UART_RXFF
# ** Warning: NUMERIC_STD."=": metavalue detected, returning FALSE
# Time: 0 ns Iteration: 0 Instance: /ariane_tb/dut/i_ariane_peripherals/gen_uart/i_apb_uart/UART_TXFF
# [TB] 0 - Asserting hard reset
# [TRACER] Output filename is: trace_hart_0.log
# [JTAG] SoftReset Done( 100)
# [TB] 5202 - init_dmi_access
# [TB] 6078 - set_dmativie
# [TB] 7830 - set_hartsel
# [TB] 7830 - Halting the Core
# Loading application to memory from /home/RISCV-2526_2/cva6-softcore-contest/sw/app/fft.mem
# [TB] 16590 - Writing the boot address into dpc
# [TB] 24474 - Resuming the CORE
# [UART]: FFT running...
# [UART]: FFT finished
# [UART]: kiss_fft took 22517 instructions and 29389 cycles
# [UART]: SUCCESS : fft result values correct

```

FIGURE 12. Log de simulation Questa — version avec optimisations logicielles (hors concours) : 22 517 instructions, **29 389 cycles**. Cette version montre le potentiel des optimisations software évoquées précédemment

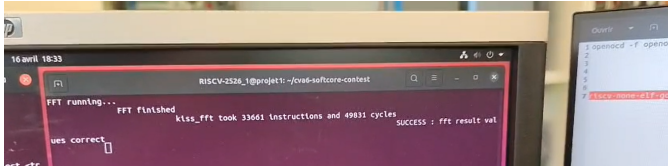


FIGURE 13. Log hyperterminal — version finale soumise au concours : 33 661 instructions, **49 831 cycles**

Les figures 14 et 15 présentent les résultats des tests sur carte de MNIST et Coremark.

```

2K performance run parameters for coremark.
CoreMark Size : 666 Total ticks : 1086033
Total time (secs): 1.080833 Iterations/Sec : 2.760314 Iterations : 3
Compiler version : GCC13.1.0 Compiler Flags : Memory location : BRAM seedcrc : 0xe9f5
[0]crc1list : 0xe714 [0]crcmatrix : 0x1fd7 [0]crcstate : 0x8e3a
[0]crcfinal : 0x2e87 Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 2.760314 / GCC13.1.0 / BRAM

```

FIGURE 14. Log hyperterminal d'un test CoreMark sur carte

```

Expected = 4
Predicted = 4
Result : 1/1 credence: 82
image env0003: 1760307 instructions
image env0003: 3086198 cycles

```

FIGURE 15. Log hyperterminal d'un test MNIST sur carte